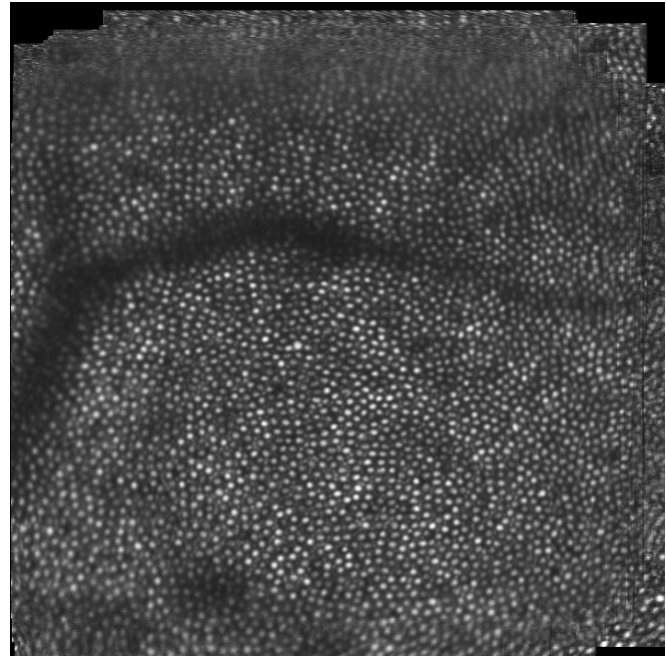
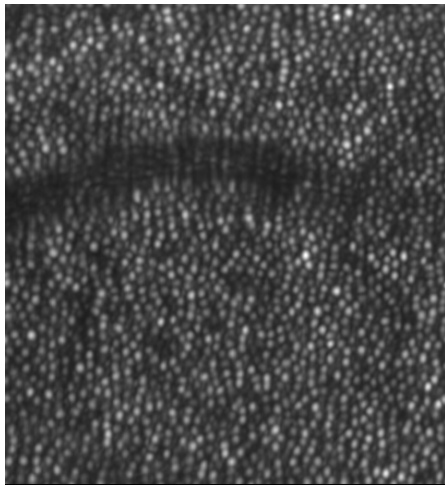
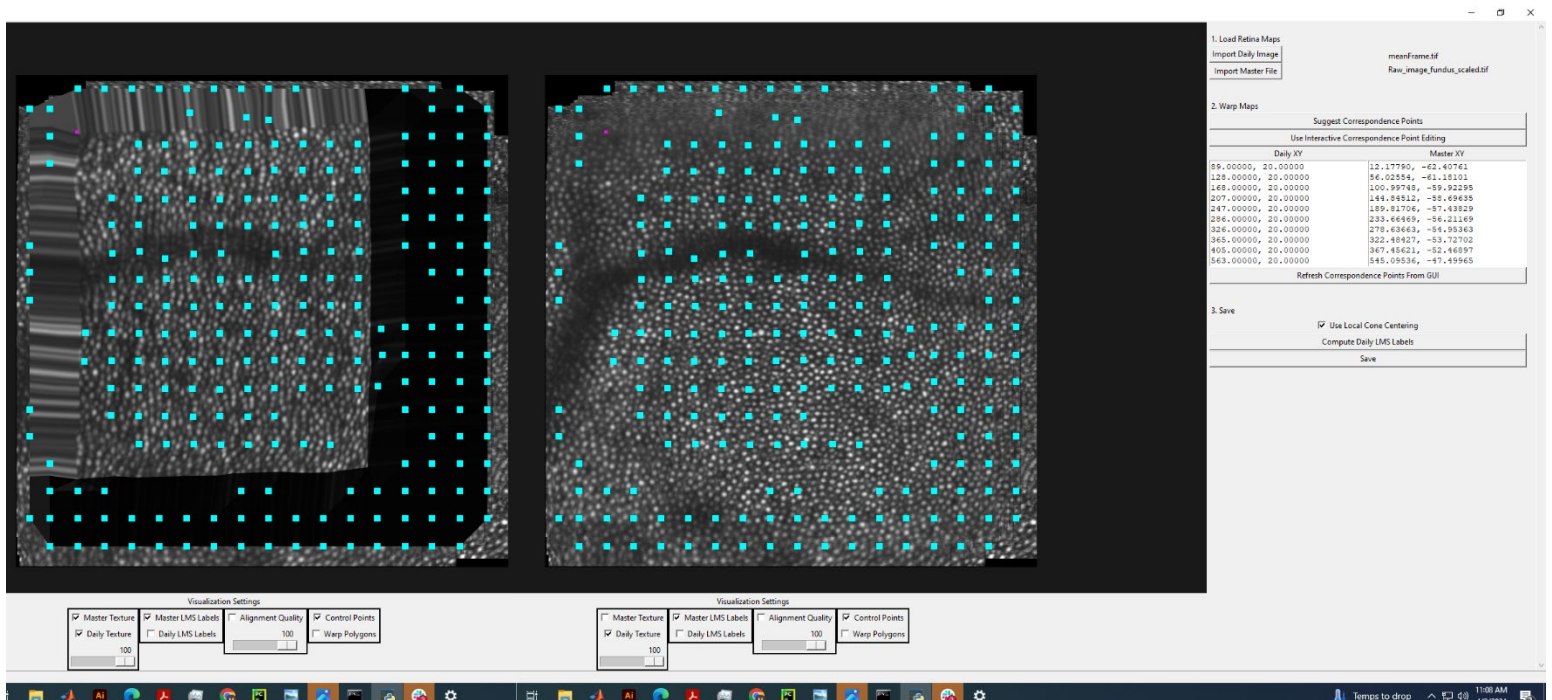


1. **Pre-processing:** I account for differences in resolution (pixels per degree) between the daily image (left) and master image (right), as well as differences in image orientation, by appropriately transforming one image so that it matches the orientation and resolution of the other. The image on left is an en face OCT image, while the image on the right is an AOSLO image (the average of many frames).



2. **Cone transfer:** I load the images into Cone Transfer Tool (CTT) and click “Suggest Correspondence Points”. Below, the warped master image is on the left and the daily image is on the right. If the transfer fails, I’ve found that it is often possible to tweak some of the hardcoded parameters in automatic_registration.py until a proper correspondence is found.



In particular, I often modify the arguments to `divide_into_patches` and `match_patches` until I get a good result.

```
# Cut map A into patches
patches = divide_into_patches(map_a, (40,40)) # was 150 150
# Find the corresponding locations on map B
match_patches(patches, map_b, (10, 10)) # was 30 30
# Do ransac to remove outliers
radius = 100; #150
local_ransac_patches(patches, radius)
```

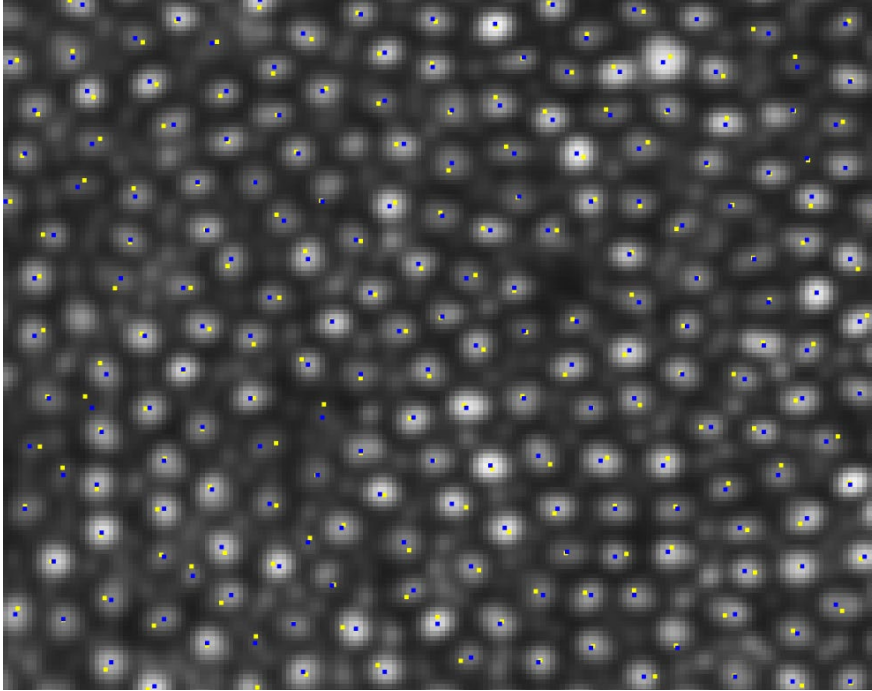
In the code above, `map_a` (the daily map) is divided into 40 x 40 pixel squares, and `match_patches` uses the `match_template` function to compare, via cross-correlation, each patch in the daily map with a patch in the master map, whose limits are determined by the third argument of `match_patches` (here (10, 10)). Finally, `local_ransac_patches` does ransac between subsets of the daily and master image as part of a function that accounts for non-affine transformations between the input images**.

** I think

3. **Centering of transferred cone locations:** CTT returns a set of corresponding points between the daily and master images. To find where any arbitrary point on the master image is on the daily image, I:
 - a. Compute the Delaunay triangulation of master correspondence points, and daily correspondence points.
 - b. Compute the affine transformation between the triangle in the master image containing the point of interest, and the corresponding triangle in the daily image.
 - c. Apply the affine transformation from (b) to the coordinates of the master point of interest.

Ultimately, I use this process to transform between cone locations in the master image and daily image, or vice-versa. However, the transferred cone locations are often slightly off from what I judge as the true cone centers. To remedy this, I use gradient ascent to shift the initial transferred cone locations to the nearest, brightest pixel, which almost always occurs at the cone center.

Below is a zoomed in portion of the daily image. The yellow markers are the initial transferred cone locations, and the blue markers are cone locations centered with gradient ascent. There are some yellow points on the very edge of a cone, for which this approach fails. To handle this, I've written an interactive script that allows the user to tweak cone locations such that the gradient ascent method works.



4. **Identification of unlabeled cones:** Not all the cones in a classified region are labeled as L, M, or S. I attempt to find some (perhaps most) of the unlabeled cones by first finding all likely cones in the image (using a simple cross-correlation based method), and then finding the difference between the set of labeled cones and the set of all likely cones. This process would be difficult without first centering the cone locations as described earlier. Below is the daily image with green markers over unlabeled cones.

